

Alight International

Multi-Vendor E-Commerce Marketplace

Document Type: Project Goals, Scope & Detailed Requirements

Version: 1.0

Project: Alight International Multi-Vendor E-Commerce Platform

Business Model: Multi-Vendor Marketplace

Primary Reference Model: Meesho-style marketplace

Backend: Java + Spring Boot

Database: PostgreSQL

1. Executive Summary

Alight International will develop a multi-vendor e-commerce marketplace that allows multiple independent vendors to sell products through a single online platform.

The platform will provide three major experiences:

1. **Customer Marketplace** — customers browse, purchase and manage products/orders.
2. **Vendor Marketplace Portal** — vendors register, manage their stores, products, inventory and orders.
3. **Alight Administration Portal** — Alight International manages vendors, customers, products, orders, payments, logistics, settlements, content and platform operations.

Alight International will act as both:

- **Marketplace Operator**
- **First-Party Seller/Vendor**

Third-party vendors will operate alongside Alight International on the same marketplace.

The customer experience should be unified, while vendor business data and operations must remain isolated.

2. Business Context

Alight International is an India-based company established in Delhi in 2017 and operates as a manufacturer, exporter, trader and supplier. Its existing business includes stainless-steel kitchen accessories, bathroom fixtures, home furnishings, wardrobe accessories and related products. The company also identifies OEM and ODM capabilities and exports worldwide.

The existing website provides company information and an e-commerce-oriented product presence.

The new platform will extend this model from a primarily single-company store into a **multi-vendor marketplace**.

3. Project Vision

The vision is to create a scalable marketplace where:

```
Customers
  ↓
Discover products from multiple vendors
  ↓
Add products from multiple vendors to one cart
  ↓
Checkout
  ↓
Make one payment
  ↓
Platform splits fulfillment internally
  ↓
Each vendor fulfills its own products
  ↓
Customer receives products
```

The platform should provide a marketplace experience similar in concept to Meesho/Flipkart while maintaining Alight International's own business identity and rules.

4. Project Goals

4.1 Primary Goals

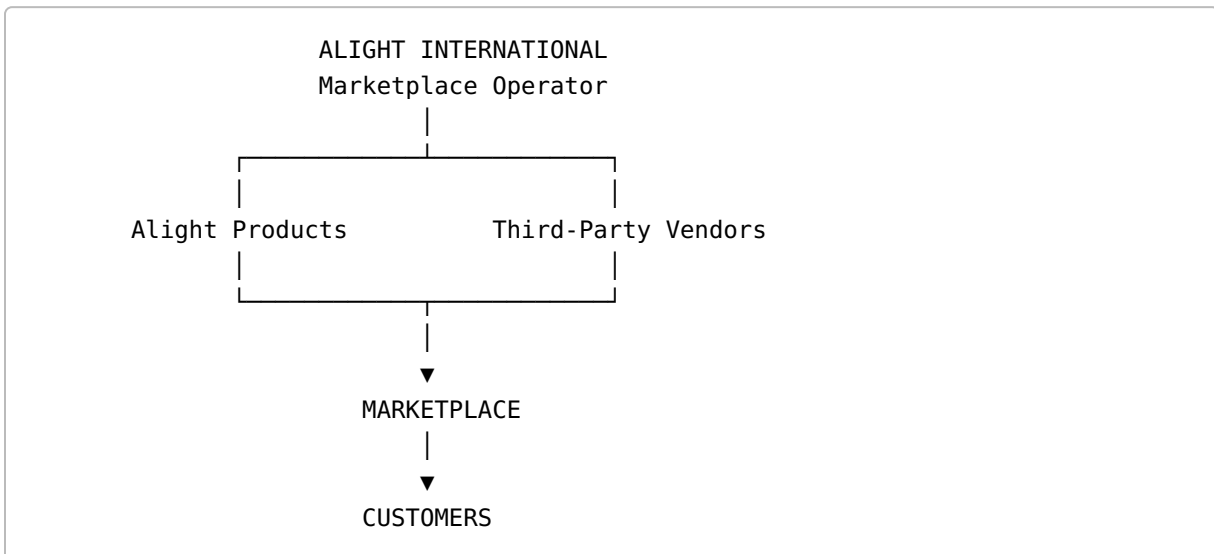
The system must:

1. Build a complete multi-vendor marketplace.
2. Allow customers to purchase products from multiple vendors.
3. Allow vendors to independently manage their products and inventory.
4. Allow Alight to approve and manage vendors.
5. Implement secure authentication.
6. Implement Role-Based Access Control.
7. Implement resource-level authorization.

8. Prevent vendors from accessing other vendors' data.
9. Support multi-vendor carts.
10. Support multi-vendor orders.
11. Support online payments.
12. Support logistics/shipping integration.
13. Support returns and refunds.
14. Support vendor settlements.
15. Support marketplace commissions or service fees.
16. Provide admin, vendor and customer dashboards.
17. Preserve useful Alight International company/product content.
18. Provide a scalable technical foundation.
19. Provide complete auditability for important operations.
20. Prepare the system for future expansion.

5. Business Model

The platform will have:



Alight itself will be represented as a first-party seller within the marketplace.

This avoids creating special business logic for Alight products.

6. User Types

The initial platform will support:

6.1 Customer

A customer purchases products from the marketplace.

6.2 Vendor

A third-party seller who has been approved by Alight.

6.3 Admin

An Alight employee responsible for marketplace operations.

6.4 Super Admin

The highest-level platform administrator.

7. Vendor Onboarding

7.1 Vendor Eligibility

Potential vendors may include:

- Manufacturers
- Wholesalers
- Distributors
- Brands
- Authorized sellers
- Other approved businesses

Final eligibility rules will be controlled by Alight.

8. Vendor Registration

Vendor registration should collect:

Personal Information

- Name
- Email
- Phone
- Password

Business Information

- Business name
- Business type
- Business address
- Pickup address
- GST information
- PAN information
- Bank information

Verification Documents

Depending on business requirements:

- GST certificate
- PAN
- Business/address proof
- Bank proof
- Authorized-person KYC
- Category-specific licenses where applicable

Documents must be securely stored and must never be publicly accessible.

9. Vendor Approval Workflow

Vendor registration:

```
graph TD; REGISTERED --> PENDING_VERIFICATION; PENDING_VERIFICATION --> UNDER_REVIEW; UNDER_REVIEW --> APPROVED;
```

Possible rejection path:

```
graph TD; UNDER_REVIEW --> REJECTED; REJECTED --> RESUBMISSION;
```

↓
UNDER_REVIEW

Approved vendors can access selling functionality.

Rejected or suspended vendors cannot sell.

10. Vendor Status

Possible vendor statuses:

PENDING
UNDER_REVIEW
APPROVED
REJECTED
SUSPENDED
BLOCKED
CLOSED

Every status transition should be recorded.

11. Customer Requirements

Customers should be able to:

- Register
- Login
- Logout
- Verify account
- Reset password
- Manage profile
- Browse products
- Search products
- Filter products
- Sort products
- View product details
- View vendors
- Add products to wishlist
- Add products to cart
- Manage addresses
- Checkout

- Pay
 - View orders
 - Track orders
 - Cancel eligible orders
 - Request returns
 - Request refunds
 - Submit reviews
 - Receive notifications
-

12. Product Catalog

The platform should support Alight's existing product domain.

Initial category candidates include:

- Modular Kitchen Accessories
- Kitchen Organizers
- Pantry Pullouts
- Multipurpose Corners
- Multipurpose Baskets
- Trays
- Waste Bins
- PVC Legs
- Kitchen Fitments
- Bathroom Fixtures
- Wardrobe Accessories
- Wardrobe Racks
- Stainless Steel Products
- Wire Products
- Flatware/Cutlery
- Tableware

The final category hierarchy must be approved by the business.

13. Product Structure

A product may contain:

- Product ID
- Vendor ID
- SKU
- Product name
- Short description

- Full description
 - Category
 - Brand
 - Images
 - Videos
 - Price
 - Discount
 - Tax information
 - Stock
 - Variants
 - Specifications
 - Weight
 - Dimensions
 - Shipping information
 - Return eligibility
 - Status
-

14. Product Lifecycle

Vendor-created products should follow:

DRAFT
↓
SUBMITTED
↓
UNDER_REVIEW
↓
APPROVED
↓
PUBLISHED

Alternative states:

REJECTED
SUSPENDED
ARCHIVED
OUT_OF_STOCK

15. Product Approval

Initially:

```
Vendor
↓
Create Product
↓
Submit
↓
Alight Admin
↓
Review
↓
Approve / Reject
```

Only approved products can become publicly available.

16. Vendor Store

Each approved vendor receives a marketplace store.

Public information may include:

- Store name
- Logo
- Store description
- Rating
- Number of products
- Product catalog
- Basic business/store information

The following must remain private:

- PAN
 - Bank account
 - KYC documents
 - Private contact information
 - Internal financial information
-

17. Vendor Dashboard

The vendor dashboard should contain:

```
Dashboard
├─ Overview
├─ Products
│   ├─ All Products
│   ├─ Add Product
│   ├─ Drafts
│   ├─ Pending Approval
│   └─ Rejected
├─ Inventory
├─ Orders
├─ Returns
├─ Reviews
├─ Earnings
├─ Settlements
├─ Store Profile
└─ Settings
```

18. Vendor Data Isolation

This is a critical security requirement.

Vendor A must never access:

- Vendor B's products
- Vendor B's inventory
- Vendor B's orders
- Vendor B's earnings
- Vendor B's customers
- Vendor B's documents
- Vendor B's settlements

Even if Vendor A manually constructs an API request.

Example:

Vendor A

```
GET /api/v1/vendors/B/products
```

↓
403 FORBIDDEN

The backend must enforce this.

19. Authentication

The platform will use secure authentication.

Recommended implementation:

```
Spring Security
+
JWT Access Token
+
Refresh Token
+
Secure Password Hashing
```

Authentication endpoints:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password
```

20. RBAC

Initial roles:

```
CUSTOMER
VENDOR
ADMIN
SUPER_ADMIN
```

Future roles may include:

```
PRODUCT_MANAGER
ORDER_MANAGER
FINANCE_MANAGER
SUPPORT_AGENT
CONTENT_MANAGER
```

Do not implement additional roles unless there is a real business requirement.

21. Authorization Model

Authorization must use both:

Role-Based Access Control

and

Resource-Level Authorization

Example:

```
Role:
VENDOR

Resource:
Product #123

Owner:
Vendor #10

Current User:
Vendor #10

Result:
ALLOW
```

If the current user is Vendor #20:

```
Result:
DENY
```

22. Shopping Cart

The cart must support products from multiple vendors.

Example:

```
CART
|
|— Vendor A
|   |— Product A
|   |— Product B
|
|— Vendor B
|   |— Product C
|
|— Alight International
|   |— Product D
```

The customer should experience this as one shopping cart.

23. Multi-Vendor Order

One checkout can create a parent customer order and multiple vendor fulfillment orders.

Example:

```
ORDER #10001
|
|— Vendor Order #10001-A
|   |— Product A
|   |— Product B
|
|— Vendor Order #10001-B
|   |— Product C
|
|— Vendor Order #10001-ALIGHT
|   |— Product D
```

Customer sees:

ORDER #10001

Vendor A sees:

Vendor Order #10001-A

Vendor B sees:

Vendor Order #10001-B

Alight Admin sees:

Complete Order #10001

24. Inventory

Inventory must track:

- Total stock
- Reserved stock
- Available stock
- Sold stock
- Returned stock
- Damaged stock

Formula:

Available Stock =
Total Stock - Reserved Stock

The system must prevent overselling.

When inventory reaches zero:

AVAILABLE
↓
OUT_OF_STOCK

The product remains in the database.

It must not be deleted automatically.

25. Order Lifecycle

Recommended initial lifecycle:

```
PENDING_PAYMENT
  ↓
PAID
  ↓
CONFIRMED
  ↓
PROCESSING
  ↓
PACKED
  ↓
READY_FOR_PICKUP
  ↓
SHIPPED
  ↓
OUT_FOR_DELIVERY
  ↓
DELIVERED
```

Alternative states:

```
CANCELLED
RETURN_REQUESTED
RETURN_APPROVED
RETURNED
REFUND_PENDING
REFUNDED
RTO
```

26. Vendor Fulfillment

The vendor is responsible for:

1. Accepting the order.
2. Preparing the product.
3. Packaging the product.
4. Printing/applying the shipping label.
5. Marking the order ready for pickup.
6. Handing the package to the logistics partner.

Meesho similarly uses a model where sellers process/pack orders and logistics partners collect and deliver them.

27. Logistics

The platform should not hard-code a single delivery company.

Instead:

```
ShippingService
├──
├── Logistics Provider A
├── Logistics Provider B
└── Logistics Provider C
```

This allows future logistics providers to be added without rewriting the order system.

Shipping calculations may depend on:

- Weight
- Dimensions
- Pickup location
- Delivery pincode
- Shipping zone
- Delivery mode
- COD/prepaid
- Logistics provider

Meesho's marketplace documentation similarly describes fulfillment costs as depending on factors such as parcel weight, distance and payment mode.

28. Shipping Zones

Initial target:

India

The platform should eventually support pincode-level serviceability.

Each product/vendor may have serviceability restrictions.

29. Returns

Initial customer return policy:

7 days from delivery for eligible products.

Meesho's current supplier shipping/returns information states that customers can return products within seven days of delivery.

Return eligibility must be configurable by category/product.

30. Non-Returnable Products

Potential exclusions:

- Customized products
- Certain hygiene-related products
- Consumable products
- Products specifically marked non-returnable
- Products excluded by applicable business policy

Final rules must be defined by Alight.

31. Return Reasons

Examples:

DAMAGED_PRODUCT
DEFECTIVE_PRODUCT
WRONG_PRODUCT
MISSING_ITEM
PRODUCT_NOT_AS_DESCRIBED
QUALITY_ISSUE
CUSTOMER_CHANGED_MIND
OTHER

Return rules can vary based on reason.

32. Return Shipping

Recommended model:

Seller/platform responsibility

For:

- Wrong product
- Defective product
- Damaged product
- Missing item
- Materially incorrect product

Customer responsibility

Potentially for eligible change-of-mind returns where the business chooses to allow them.

The exact policy must be configurable.

Meesho currently charges sellers return shipping fees based on shipment weight in its supplier policy.

33. RTO

RTO means:

Return to Origin

Example:

```
Vendor
↓
Courier
↓
Customer unavailable/refuses
↓
Delivery attempts fail
↓
RTO
↓
Vendor
```

The system must track RTO separately from customer returns.

34. Payment Gateway

Initial recommended payment gateway:

Razorpay

The payment module must support:

- Payment creation
- Payment initiation
- Payment callback/webhook
- Signature verification
- Payment success
- Payment failure
- Refund
- Transaction reconciliation

The backend must never trust the frontend's payment-success response.

35. Payment Architecture

```
Customer
↓
Checkout
↓
Spring Boot
↓
```

```
Payment Gateway
↓
Payment
↓
Webhook
↓
Spring Boot verifies payment
↓
Order marked PAID
```

36. Cash on Delivery

If COD is enabled, the system should support:

- COD eligibility
- COD order creation
- COD shipping
- Delivery confirmation
- Payment collection status
- COD reconciliation

COD should be configurable by:

- Product
- Vendor
- Pincode
- Order value
- Logistics provider

37. Vendor Commission / Marketplace Fees

Unlike Meesho, Alight should not automatically copy a 0% commission model.

The platform must support configurable fees.

Possible model:

```
Product Sale
↓
Gross Sale Amount
↓
```

Marketplace Fee
↓
Shipping/Logistics Fees
↓
Taxes/Applicable Charges
↓
Vendor Payable

Example only:

Sale: ₹1,000

Marketplace Fee: ₹100
Shipping: ₹60
Other applicable deductions: ₹20

Vendor Settlement: ₹820

Actual rates must be configurable by Alight.

38. Settlement

Recommended settlement flow:

CUSTOMER PAYMENT
↓
ORDER DELIVERED
↓
RETURN WINDOW
↓
SETTLEMENT ELIGIBLE
↓
CALCULATE FEES
↓
CALCULATE VENDOR PAYABLE
↓
SETTLEMENT
↓
PAID

Possible settlement states:

PENDING
ELIGIBLE
PROCESSING
PAID
FAILED
ON_HOLD
REVERSED

The system must maintain a settlement ledger.

39. Meesho-Inspired Payment Cycle

Meesho currently publicly describes a 7-day payment cycle after delivery for suppliers.

For Alight, the initial configurable policy should be:

Settlement after successful delivery + applicable return/hold period.

The exact number of days should be configurable by Admin.

This avoids hard-coding a business policy into the application.

40. Discounts

Vendors may create product-level discounts subject to marketplace rules.

Supported discount types:

- Percentage
- Fixed amount
- Product-specific
- Category-specific
- Vendor-specific
- Time-limited

Admin should have the ability to:

- Approve promotions
- Set maximum discount limits
- Disable promotions
- Create marketplace-wide campaigns

41. Coupons

Coupon capabilities:

- Coupon code
- Discount type
- Discount value
- Minimum order
- Maximum discount
- Start date
- End date
- Usage limit
- Customer usage limit
- Product restrictions
- Category restrictions
- Vendor restrictions

42. Reviews

Customers should be able to review eligible purchased products.

Review fields:

- Rating
- Review text
- Images
- Date
- Verified purchase indicator

Admin should be able to moderate reviews.

43. Wishlist

Customers can:

- Add products
 - Remove products
 - View wishlist
 - Move product to cart
-

44. Search

Search should support:

- Product name
- SKU
- Brand
- Category
- Vendor
- Attributes

Future scope:

- Full-text search engine
 - Typo tolerance
 - Search suggestions
 - Ranking
 - Personalized search
-

45. Product Filtering

Initial filters:

- Category
- Price
- Brand
- Vendor
- Rating
- Availability
- Attributes

Sorting:

- Relevance
 - Price low to high
 - Price high to low
 - Newest
 - Rating
 - Popularity
-

46. Customer Data

Alight should control the marketplace customer relationship and platform data subject to applicable law and contracts.

Vendors should receive only information required to fulfill their portion of an order.

Vendors must not receive unnecessary customer information.

The platform must follow applicable privacy and data-protection requirements.

47. Admin Dashboard

Admin dashboard should provide:

Dashboard

└ Customers

└ Vendors

└ Products

└ Categories

└ Inventory

└ Orders

└ Payments

└ Returns

└ Refunds

└ Settlements

└ Coupons

└ Reviews

└ Content

└ Reports

└ Audit Logs

48. Vendor Management

Admin can:

- View vendors
- Search vendors
- Approve vendors
- Reject vendors

- Suspend vendors
 - Reactivate vendors
 - View documents
 - View products
 - View vendor orders
 - View vendor settlements
 - View vendor performance
-

49. Customer Management

Admin can:

- View customers
- Search customers
- View order history
- Suspend accounts
- Reactivate accounts
- Handle customer issues

Admins should not be able to access sensitive authentication secrets.

50. Content Management

The existing Alight website contains useful business content such as:

- About Us
- Company information
- FAQs
- Blogs
- Contact information

The new platform should retain and improve this content. Alight's existing site identifies the company as a manufacturer/exporter/trader/supplier and lists its major product domains.

Admin should be able to manage:

- Pages
- FAQs
- Blogs
- Banners
- Homepage sections
- Policies
- Promotional content

51. Customer Support

Support should be primarily managed by Alight.

Customer should be able to raise:

- Order issue
- Payment issue
- Delivery issue
- Return request
- Refund issue
- Product issue
- Account issue

Vendor support should be separate from customer support.

52. Notifications

Customer

- Registration
- Verification
- Order confirmation
- Payment confirmation
- Order processing
- Shipment
- Delivery
- Return
- Refund

Vendor

- Vendor approval
- Product approval
- New order
- Return request
- Settlement
- Account changes

Admin

- New vendor
- Product awaiting approval

- Payment issue
 - Return request
 - Settlement issue
 - Security event
-

53. Audit Logging

Important operations must be logged.

Examples:

```
Admin approved vendor
Admin rejected vendor
Admin suspended vendor
Vendor created product
Vendor changed product price
Vendor updated inventory
Customer placed order
Admin refunded order
Admin changed commission
Admin changed settlement
```

Audit records should contain:

- Actor
 - Action
 - Resource
 - Resource ID
 - Timestamp
 - Result
 - Relevant metadata
-

54. Security Requirements

The system must implement:

- Secure password hashing
- JWT security
- Refresh-token security
- Role-based authorization
- Resource ownership authorization
- Input validation

- SQL injection protection
- XSS protection
- CSRF protection where applicable
- Rate limiting
- Secure HTTP headers
- Secure file uploads
- Secrets management
- API authentication
- API authorization
- Audit logging

Sensitive business operations must always be executed server-side.

55. File Upload Security

Vendor/product uploads must validate:

- File type
- File extension
- MIME type
- File size
- Filename
- Image dimensions where appropriate

Uploaded files must not be executable.

Private documents must require authenticated authorization.

56. Database Requirements

Primary database:

PostgreSQL

Major entities:

```
users
roles
permissions
user_roles

vendors
vendor_documents
```

vendor_addresses

products
product_variants
product_images
categories
brands

inventory
inventory_transactions

carts
cart_items

orders
order_items
vendor_orders

payments
payment_transactions

addresses

coupons
coupon_usage

reviews

returns
refunds

settlements
settlement_transactions

notifications

audit_logs

content_pages
faqs
blogs
banners

The final ERD must be created before implementation of the full system.

57. Backend Architecture

The backend should initially be a **modular monolith**.

```
Spring Boot
|
├─ auth
├─ users
├─ vendors
├─ products
├─ categories
├─ inventory
├─ cart
├─ orders
├─ payments
├─ settlements
├─ returns
├─ refunds
├─ reviews
├─ coupons
├─ logistics
├─ notifications
├─ content
└─ audit
```

Microservices should not be introduced during MVP unless there is an actual scaling or organizational requirement.

58. REST API

API base path:

```
/api/v1
```

Major API groups:

```
/api/v1/auth
/api/v1/users
/api/v1/vendors
/api/v1/products
/api/v1/categories
```

```
/api/v1/cart  
/api/v1/orders  
/api/v1/payments  
/api/v1/inventory  
/api/v1/settlements  
/api/v1/returns  
/api/v1/refunds  
/api/v1/reviews  
/api/v1/coupons  
/api/v1/logistics  
/api/v1/notifications  
/api/v1/admin
```

All protected endpoints must enforce authentication and authorization.

59. API Standards

APIs should use:

- REST conventions
 - HTTP status codes
 - Request validation
 - Response DTOs
 - Pagination
 - Filtering
 - Sorting
 - Centralized exception handling
 - Consistent error responses
 - API versioning
 - Authentication
 - Authorization
 - OpenAPI/Swagger documentation
-

60. Recommended Technology Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Backend

- Java
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate

Database

- PostgreSQL

Cache

- Redis

Testing

- JUnit 5
- Mockito
- Spring Boot Test
- Testcontainers
- Postman

Infrastructure

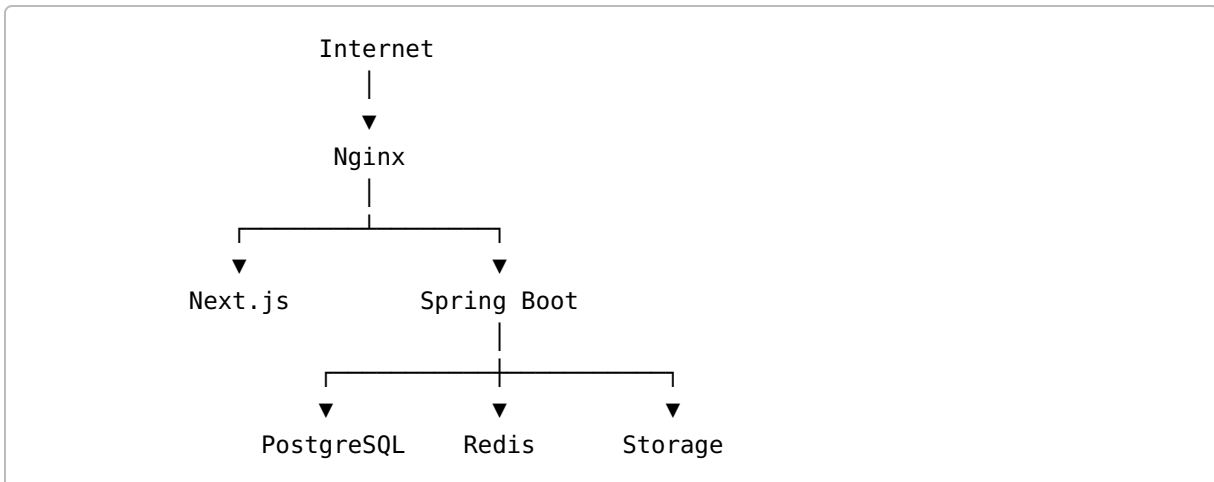
- Docker
- Docker Compose
- Nginx
- Git
- GitHub
- GitHub Actions

External Services

- Razorpay
- Logistics/shipping provider
- Object storage
- Email service
- SMS service where required

61. Deployment Architecture

Initial deployment:



Everything should be containerized where practical.

62. Testing Requirements

Unit Testing

Test:

- Services
- Business rules
- Calculations
- Validation
- Authorization logic

Integration Testing

Test:

- Database interactions
- Authentication
- Order creation
- Payment processing
- Inventory
- Vendor isolation

API Testing

Use Postman or automated API tests.

Important security tests:

```
Vendor A → Vendor B product
Vendor A → Vendor B order
Customer → Admin API
Customer → Vendor API
Vendor → Admin API
```

All unauthorized requests must fail.

63. Non-Functional Requirements

Performance

The system should:

- Use pagination.
- Avoid unnecessary database queries.
- Use caching where appropriate.
- Optimize images.
- Avoid loading unnecessary data.

Scalability

The architecture should support growth in:

- Customers
- Vendors
- Products
- Orders
- Traffic

Availability

Production should include:

- Database backups
 - Health checks
 - Monitoring
 - Logging
 - Error tracking
 - Recovery procedures
-

64. MVP

The first production MVP should include:

Customer

- Registration/login
- Product browsing
- Search
- Product details
- Cart
- Address
- Checkout
- Payment
- Orders

Vendor

- Registration
- Verification
- Admin approval
- Vendor dashboard
- Product management
- Inventory
- Orders

Admin

- Dashboard
- Vendor management
- Customer management
- Product approval
- Category management
- Order management
- Payment visibility

Platform

- Authentication
- RBAC
- Vendor isolation
- PostgreSQL
- REST APIs
- Basic notifications
- Docker
- Logging

65. Phase 2

After MVP:

- Returns
 - Refunds
 - Vendor settlements
 - Coupons
 - Reviews
 - Advanced inventory
 - Logistics integration
 - Vendor analytics
 - Customer support
 - Content management
-

66. Phase 3

Future capabilities:

- Advanced analytics
 - Recommendation engine
 - Advanced search
 - Vendor advertising
 - Loyalty system
 - Promotional campaigns
 - Multiple logistics providers
 - Automated settlements
 - Mobile application
 - Multi-language support
 - Multi-currency support
 - Advanced fraud detection
-

67. Business Decisions Requiring Final Approval

Before production, Alight must finalize:

1. Vendor eligibility.
2. Required documents.
3. Vendor approval process.
4. Marketplace commission/service-fee model.
5. Settlement cycle.

6. Shipping responsibility.
7. Logistics partners.
8. Return window.
9. Return shipping responsibility.
10. Non-returnable categories.
11. Payment gateway.
12. GST/tax treatment.
13. Customer-data policy.
14. Public vendor information.
15. Alight's first-party seller status.
16. Vendor discount rules.
17. Product approval rules.
18. Out-of-stock behavior.
19. Cancellation rules.
20. Settlement rules.
21. Shipping zones.
22. Customer-support process.

68. Initial Business Policy

For the MVP, the recommended default policy is:

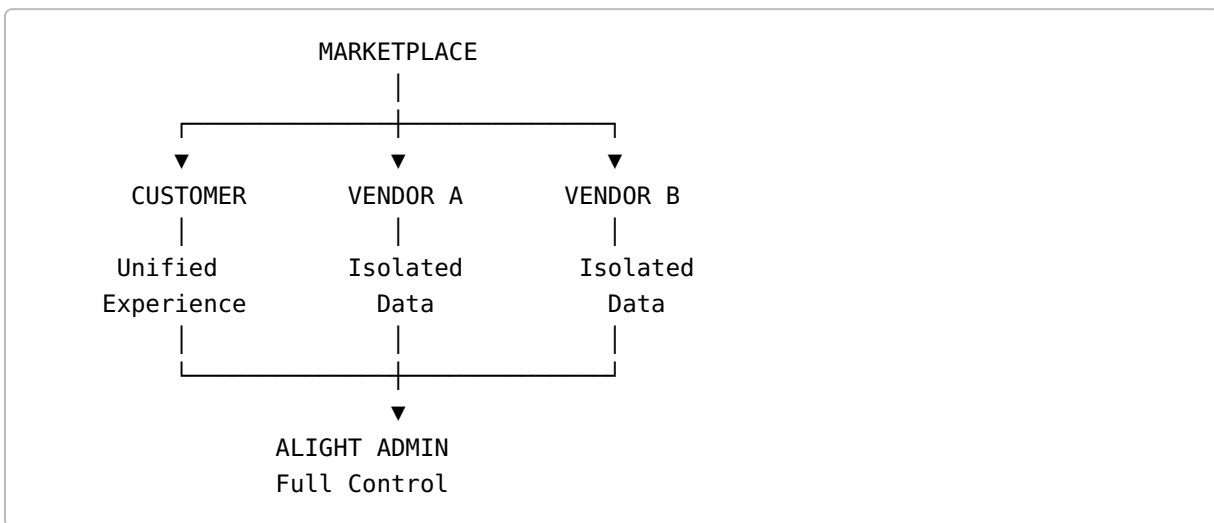
Area	Initial Policy
Vendor eligibility	Verified businesses/sellers
Vendor approval	Alight Admin
Vendor documents	GST/PAN/business/bank/KYC as applicable
Commission	Configurable
Settlement	Configurable after delivery/return window
Shipping	Platform-coordinated
Packaging	Vendor
Pickup	Logistics partner
Delivery	Logistics partner
Multi-vendor order	Supported
Return period	7 days for eligible products
Return shipping	Based on reason/policy
Payment	Razorpay initially

Area	Initial Policy
Taxes	GST-aware; final rules validated by CA
Customer data	Platform controlled, minimum vendor access
Alight products	First-party seller
Vendor discounts	Allowed within platform rules
Product approval	Admin initially
Zero stock	Product becomes unavailable
Cancellation	Allowed according to order state
Shipping coverage	India initially
Support	Centralized Alight support

69. Core Marketplace Principle

The most important architectural rule is:

One marketplace, unified customer experience, isolated vendor operations.



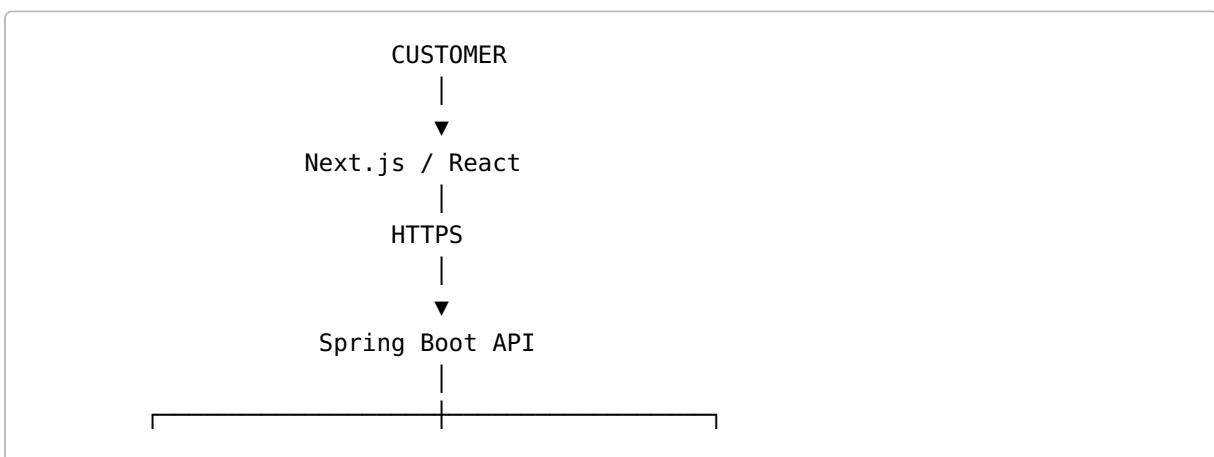
70. Definition of Done

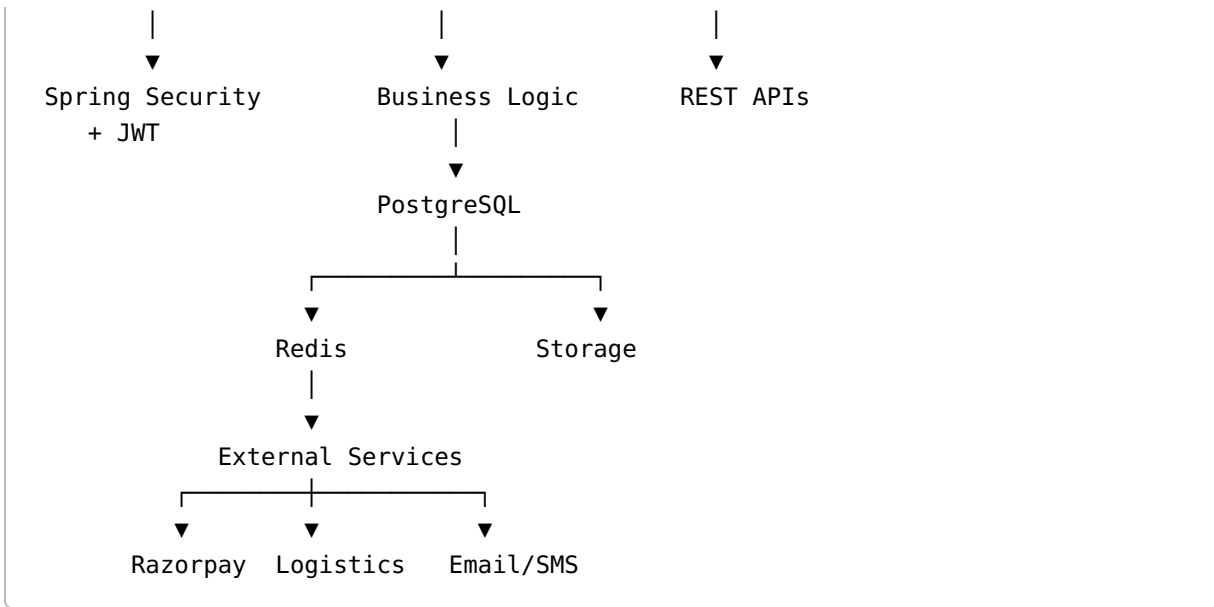
The MVP will be considered complete when:

- A customer can register and authenticate.
- A vendor can register.
- Admin can approve a vendor.
- Approved vendors can create products.
- Admin can approve products.
- Customers can browse products.
- Customers can add products from multiple vendors to a cart.
- Customers can checkout.
- Payment can be securely verified.
- A parent order can be created.
- Vendor-specific orders can be generated.
- Vendors can see only their own orders.
- Vendors can update fulfillment status.
- Inventory is correctly reserved and deducted.
- Customers can see their complete order.
- Admin can see the entire marketplace order.
- Unauthorized access is blocked.
- Vendor data is isolated.
- Payment records are stored.
- Audit logs are generated for important operations.
- The system can be deployed using Docker.
- Core APIs have automated tests.
- Production configuration and secrets are separated from source code.

71. Final Architecture Direction

The project should initially follow:





The backend should remain a **modular monolith** initially.

The system should be designed so that individual modules can later be extracted into services if the business reaches a scale where that is justified.

72. Development Principle

The project should be developed in this order:

1. Requirements
↓
2. Database / ERD
↓
3. RBAC & Permission Matrix
↓
4. API Specification
↓
5. Backend Architecture
↓
6. Authentication
↓
7. Vendor Management
↓
8. Product Catalog
↓
9. Inventory
↓

10. Cart
- ↓
11. Multi-Vendor Orders
- ↓
12. Payments
- ↓
13. Logistics
- ↓
14. Returns / Refunds
- ↓
15. Settlements
- ↓
16. Admin Dashboard
- ↓
17. Vendor Dashboard
- ↓
18. Customer Frontend
- ↓
19. Testing
- ↓
20. Docker / Deployment

Do not start by building the UI and then inventing the backend around it.

The marketplace's hardest parts are the **authorization model, multi-vendor order structure, inventory, payment state, returns and settlements**. Those should be designed before implementation.